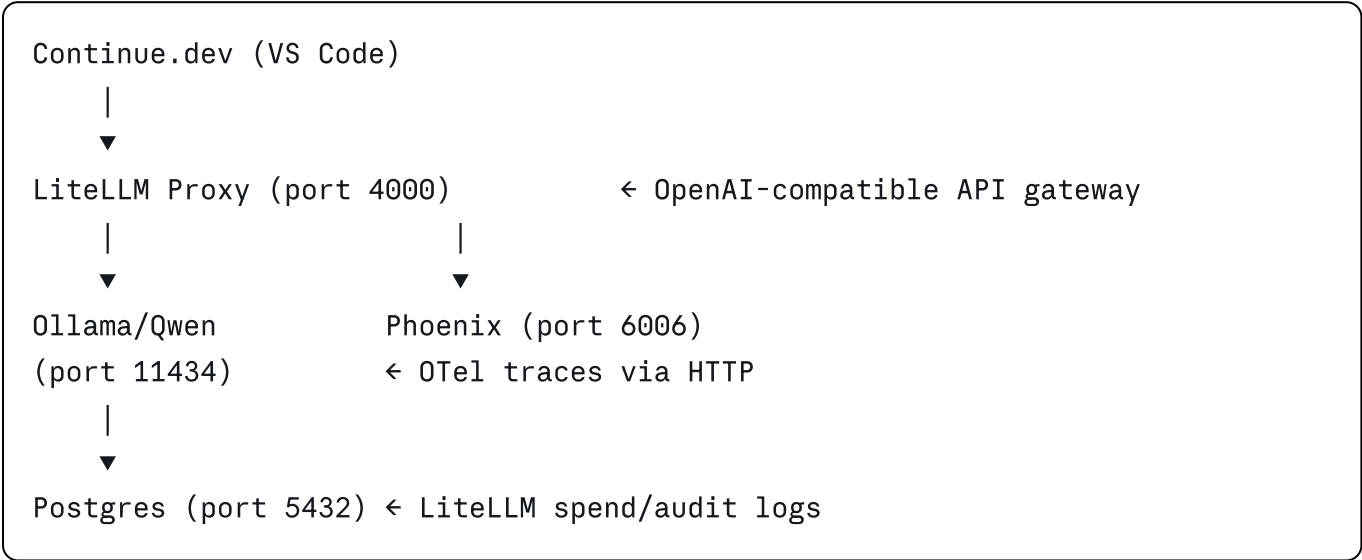


Local LLM Orchestrator — Installation Guide

A local AI development stack running Ollama (Qwen LLM) + LiteLLM (proxy/gateway) + Arize Phoenix (observability), with full OpenTelemetry tracing and Continue.dev integration for VS Code.

Architecture Overview



Prerequisites

Required Software

Software	Version	Notes
Python	3.11.x	Must be 3.11 — LiteLLM + Prisma is unstable on 3.12/3.13 on Windows
Docker Desktop	Latest	For running Postgres
Ollama	Latest	For running local LLMs
Git	Latest	

Windows-Specific Requirements

- Developer Mode must be ON — required for Prisma to create symlinks without admin rights
 - Settings → Privacy & Security → For Developers → Developer Mode → ON
- PowerShell execution policy — scripts set this per-session automatically via `ExecutionPolicy Bypass`

Python Installation

Download Python 3.11 from <https://www.python.org/downloads/release/python-3119>

During installation:

- ☒ Check "Add Python 3.11 to PATH"
- ☒ Check "Install for all users"
- ☒ Check **pip** and **py launcher** under Customize Installation

Verify:

```
powershell

python --version    # Python 3.11.x
pip --version       # pip XX from ... (python 3.11)
```

Postgres Setup (Docker)

Postgres runs as a standalone Docker container **outside** the orchestrator scripts — start it manually before running the stack.

```
bash

docker run -d \
  --name postgres-litellm \
  -e POSTGRES_PASSWORD=Password123 \
  -e POSTGRES_DB=litellm \
  -p 5432:5432 \
  postgres:latest
```

Verify it's running:

```
powershell

docker ps | findstr postgres
Test-NetConnection -ComputerName 127.0.0.1 -Port 5432
```

Important: Use `127.0.0.1` not `localhost` in all connection strings. On Windows, `localhost` can resolve to IPv6 (`:::1`) which Docker doesn't always bind to.

Ollama Setup

1. Download and install Ollama from <https://ollama.com>

2. Pull the Qwen model:

powershell

```
ollama pull qwen2.5-coder:0.5b
```

Verify Ollama is running:

powershell

```
Invoke-RestMethod -Uri "http://localhost:11434/api/tags"
```

Project Structure

```

local_llm_orchestrator\
├─ start_orchestrator.bat           ← Main entry point
├─ shutdown_orchestrator.bat       ← Teardown
├─ requirements-litellm.txt         ← LiteLLM venv packages
├─ requirements-phoenix.txt        ← Phoenix venv packages
├─ INSTALL.md                      ← This file
├─ config\
│   ├── config.yaml               ← LiteLLM proxy configuration
│   └─ env.ps1                   ← All environment variables
├─ bin\
│   ├── Start-Stack.ps1          ← Orchestrator hub
│   ├── Stop-Stack.ps1           ← Shutdown engine
│   ├── Run-LiteLLM.ps1          ← LiteLLM spoke
│   └─ Run-Phoenix.ps1           ← Phoenix spoke
├─ env\
│   ├── .venv\                  ← LiteLLM virtual environment (Python
3.11)
│   └─ .venv-phoenix\           ← Phoenix virtual environment (Python
3.11)
├─ log\                          ← Auto-generated logs
│   ├── litellm_combined.log
│   ├── phoenix_combined.log
│   └─ orchestrator_start.log
└─ tmp\                          ← Auto-generated PID files

```

Virtual Environment Setup

Why Two Separate venvs?

LiteLLM and Arize Phoenix have **conflicting dependencies** — specifically around `mcp`, `fastapi`, `rich`, and `cryptography` versions. Running them in the same venv causes import errors. The solution is two isolated environments:

- `.venv` — LiteLLM proxy (pinned to a stable version)
- `.venv-phoenix` — Arize Phoenix (latest)

Step 1 — Create venvs

```
powershell

# From the project root
cd D:\llm_project\local_llm_orchestrator

# LiteLLM venv
python -m venv env\.venv

# Phoenix venv
python -m venv env\.venv-phoenix
```

Step 2 — Install LiteLLM

```
powershell

# Activate LiteLLM venv
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process -Force
.\env\.venv\Scripts\Activate.ps1

# Upgrade pip
python -m pip install --upgrade pip

# Install LiteLLM (pinned — see version note below)
pip install "litellm[proxy]==1.72.6"

# Install Prisma (required by LiteLLM for Postgres)
pip install prisma

# Generate Prisma client (downloads Node.js internally — requires Developer Mod
$schema = python -c "import litellm, os; print(os.path.join(os.path.dirname(lit
prisma generate --schema $schema

deactivate
```

⚠ **LiteLLM Version Warning:** Pin to `1.72.6`. Versions 1.90.x introduced a breaking change where Prisma startup failures cause **immediate silent shutdown** instead of logging a warning and continuing. Do not upgrade without testing — specifically verify `DATABASE_URL` is loaded before LiteLLM starts, otherwise Prisma engine crashes silently.

Step 3 — Install Phoenix

```
powershell

# Install Phoenix in its own isolated venv
.\env\.venv-phoenix\Scripts\pip.exe install arize-phoenix
```

Verify Phoenix starts:

```
powershell

.\env\.venv-phoenix\Scripts\phoenix.exe serve --port 6006
# Should show: Uvicorn running on http://0.0.0.0:6006
# Press Ctrl+C to stop
```

Configuration

`config\env.ps1`

All environment variables are centralized here. The scripts source this automatically — you never need to load it manually.

```
powershell
```

```
# Encoding
$env:PYTHONIOENCODING = "utf-8"
$env:LC_ALL = "en_US.UTF-8"

# Ports
$env:PHOENIX_PORT = 6006
$env:LITELLM_PORT = 4000

# Paths
$WorkspaceRoot = Split-Path -Parent $PSScriptRoot
$env:LITELLM_CONFIG = "$WorkspaceRoot\config\config.yaml"

# Keys
$env:OPENAI_API_KEY = "sk-mock-key-for-local-routing-boundaries"

# Database – use 127.0.0.1, NOT localhost
$env:DATABASE_URL = "postgresql://postgres:Password123@127.0.0.1:5432/litellm?p

# Prisma engine – path from prisma generate output
$env:PRISMA_QUERY_ENGINE_BINARY = "C:\Users\<username>\.cache\prisma-python\bin
$env:PRISMA_QUERY_ENGINE_BINARY_TIMEOUT = "60"
$env:PRISMA_CLIENT_CONNECT_TIMEOUT = "30"
$env:PRISMA_CLIENT_ENGINE_TYPE = "binary" # must be quoted string

# OpenTelemetry – LiteLLM uses its own env vars (NOT standard OTEL_EXPORTER_OTL
$env:OTEL_EXPORTER = "otlp_http"
$env:OTEL_ENDPOINT = "http://127.0.0.1:6006/v1/traces"

$env:LITELLM_LOG = "INFO"
```

Critical: `DATABASE_URL` **must** be set before LiteLLM starts. The Prisma query engine reads this on startup and exits silently if it's missing. This is handled automatically by `env.ps1` being sourced in every script.

`config\config.yaml`

yaml

```
model_list:
  - model_name: qwen-local
    litellm_params:
      model: ollama/qwen2.5-coder:0.5b
      api_base: http://localhost:11434
      drop_params: true      # drops unsupported params like parallel_tool_calls

litellm_settings:
  callbacks: ["otel"]      # enables OpenTelemetry tracing to Phoenix

general_settings:
  master_key: sk-master-1234
  database_connection_pool_limit: 10
  database_connection_timeout: 60
  disable_spend_logs: false
  health_check_interval: 120
  use_client_credentials_from_environment_variables: true
  database_url: "postgresql://postgres:Password123@127.0.0.1:5432/litellm?pool_
```

OTel Note: LiteLLM uses its own custom env vars `OTEL_EXPORTER` and `OTEL_ENDPOINT` — not the standard `OTEL_EXPORTER_OTLP_ENDPOINT`. The callback key is `callbacks: ["otel"]` under `litellm_settings` — not `litellm_logging` under `general_settings`.

Running the Stack

Start

```
bat
start_orchestrator.bat
```

This will:

1. Load all environment variables from `config\env.ps1`
2. Start Phoenix in background (`.venv-phoenix`)
3. Wait for Phoenix readiness on port 6006 (up to 80 seconds)
4. Start LiteLLM in background (`.venv`)
5. Wait 20 seconds for database migrations
6. Wait for LiteLLM readiness on port 4000
7. Report `ALL SYSTEMS GO` when both are healthy

Stop

```
bat  
  
shutdown_orchestrator.bat
```

Cleanly terminates all process trees.

Verify

```
powershell  
  
# LiteLLM health  
Invoke-RestMethod -Uri "http://localhost:4000/health"  
  
# Test a request  
$body = @{  
    model = "qwen-local"  
    messages = @( @{role="user"; content="Say hello"} )  
} | ConvertTo-Json  
  
Invoke-RestMethod -Uri "http://localhost:4000/chat/completions" `  
    -Method POST `  
    -Headers @{ "Authorization"="Bearer sk-master-1234"; "Content-Type"="application/json" } `  
    -Body $body
```

Continue.dev Integration (VS Code)

Install the Continue.dev extension, then configure `~/.continue/config.yaml`:

```
yaml
```



```
name: Main Config
version: 1.0.0
schema: v1
models:
  - name: qwen-local-no-trace      # Direct to Ollama, no observability
    provider: ollama
    model: qwen2.5-coder:0.5b
    capabilities:
      - tool_use

  - name: qwen-local              # Through LiteLLM, full Phoenix tracing
    provider: openai              # OpenAI-compatible format
    model: qwen-local
    apiBase: http://localhost:4000
    apiKey: sk-master-1234
    capabilities:
      - tool_use
```

Note: `provider: openai` does not mean OpenAI's cloud service — it tells Continue.dev to use the OpenAI API format, which LiteLLM exposes. The `apiBase` overrides the destination to your local LiteLLM instance.

Observability

- **Phoenix UI:** `http://localhost:6006` — view all LLM traces, spans, token usage
- **LiteLLM UI:** `http://localhost:4000/ui` — view spend logs, API keys, model usage
- **Logs:** `log\litellm_combined.log` and `log\phoenix_combined.log`

Traces appear in Phoenix automatically for every request routed through LiteLLM (`qwen-local` model). Direct Ollama requests (`qwen-local-no-trace`) are not traced.

Troubleshooting

LiteLLM shuts down immediately after starting

Cause: `DATABASE_URL` not set in environment before startup. **Fix:** Ensure `env.ps1` is sourced before running LiteLLM. The scripts do this automatically — if running manually, run `.\config\env.ps1` first.

Phoenix fails readiness probe

Cause: Phoenix takes 20-30 seconds to initialize on cold start. **Fix:** The readiness probe allows up to 80 seconds (20 retries × 4 seconds). If it still fails, check

`log\phoenix_combined.log` for import errors.

`parallel_tool_calls` unsupported error

Cause: Continue.dev sends OpenAI-specific parameters that Ollama doesn't support. **Fix:** `drop_params: true` in the model's `litellm_params` in `config.yaml`.

No traces in Phoenix

Cause: Wrong OTel env var names or endpoint. **Fix:** LiteLLM uses `OTEL_EXPORTER=otlp_http` and `OTEL_ENDPOINT=http://127.0.0.1:6006/v1/traces` — not the standard `OTEL_EXPORTER_OTLP_ENDPOINT`. Verify these are set in `env.ps1`.

Prisma symlink error during `prisma generate`

Cause: Windows requires admin rights for symlinks by default. **Fix:** Enable Developer Mode in Windows Settings.

LiteLLM UI login fails with `policies column does not exist`

Cause: Database schema mismatch — created by a different LiteLLM version. **Fix:** Drop and recreate the Postgres database, then restart the stack to run fresh migrations.

```
powershell

docker exec -it <postgres-container> psql -U postgres -c "DROP DATABASE litellm"
docker exec -it <postgres-container> psql -U postgres -c "CREATE DATABASE litellm"
.\start_orchestrator.bat
```

Version Reference

Component	Version	Notes
Python	3.11.9	Required — 3.12/3.13 has Prisma issues on Windows
litellm	1.72.6	Pinned — 1.90.x breaks on startup failure handling
arize-phoenix	Latest	Installed in separate venv
prisma	0.15.0	Auto-installed with litellm
Postgres	Latest	Via Docker
Ollama	Latest	
qwen2.5-coder	0.5b	Lightweight coding model